INFORME DE ACTIVIDADES DESARROLLADAS

Práctica 05

Generación de pruebas unitarias mediante cobertura de código

CURSO	DOCENTE	PROGRAMA
Ingeniería de Software II	DSc. Edgar Sarmiento Calisaya	Ciencia de la Computación

Objetivo de la práctica

Diseñar casos de prueba para el algoritmo bubbleSort utilizando su grafo de flujo de control y una estrategia de cobertura de decisiones más condiciones simples. La solución sigue las cuatro actividades especificadas en la guía proporcionada.

Síntesis de resultados

GRAFO DE CONTROL	COMPLEJIDAD	VERIFICACIÓN
9 nodos · 11 aristas	V(G) = 4	4 casos de prueba + 4 adicionales
3 nodos de decisión	4 caminos independientes	6/6 ramas cubiertas · 8/8 tests OK

1. Código base y criterio de interpretación

El enunciado presenta un método de ordenamiento ascendente por burbuja con dos bucles anidados y una comparación que decide si intercambiar elementos consecutivos.

```
bubbleSort(int vet[]) {
    integer i, j, aux;
    for (i=tamanho(vet); i>=2; i--) {
        for (j=1; j<=i-1; j++) {
            if (vet[j] > vet[j+1]) {
                aux=vet[j];
                vet[j]=vet[j+1];
                vet[j+1]=aux;
            }
        }
    }
}
```

Supuesto necesario: el pseudocódigo usa índices desde 1. La implementación ejecutable en Python ajusta el acceso a índices desde 0 sin cambiar el comportamiento ni las decisiones evaluadas.

2. Actividad 1 | Grafo de flujo de control

El grafo distingue las tres decisiones (rombos) y los dos retornos de los bucles. V representa una evaluación verdadera y F una falsa. Se identifican nueve nodos y once aristas visibles.

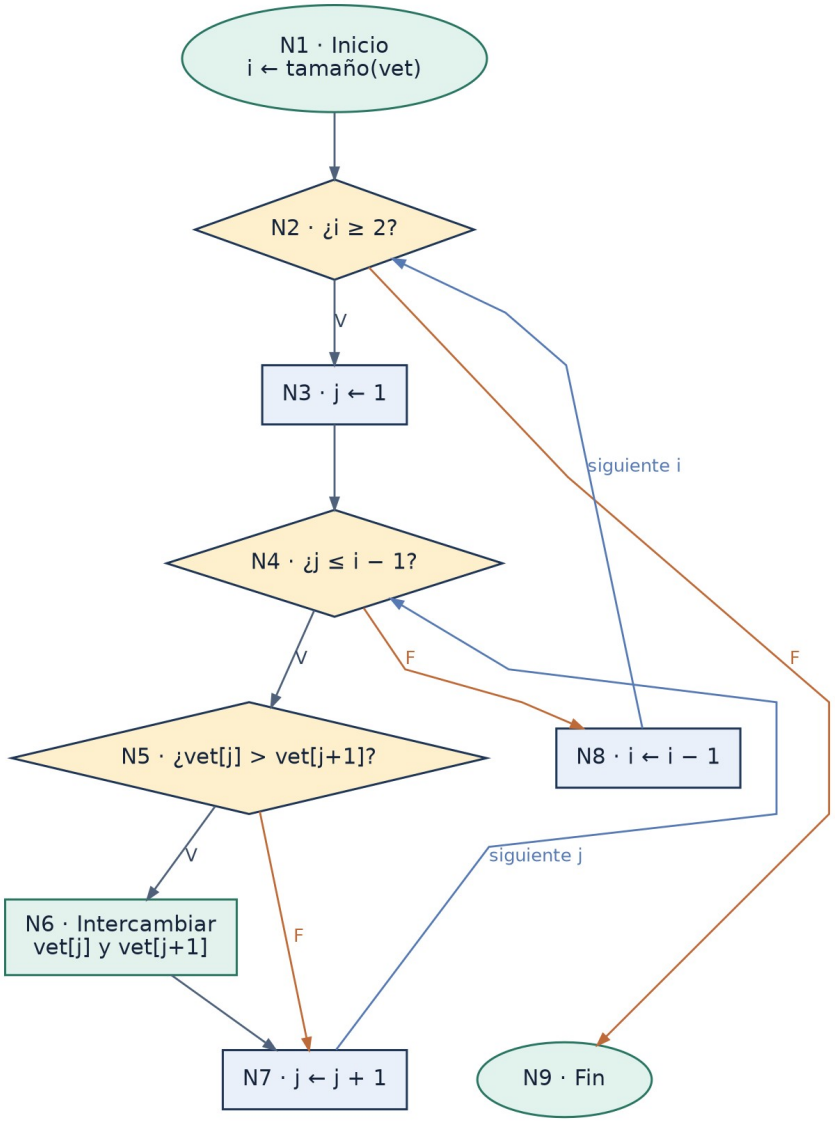


Figura 1. Grafo de flujo de control para el bubbleSort del enunciado. Fuente: elaboración a partir de la guía de la práctica.

NODO	OPERACIÓN	NODO	OPERACIÓN
N1	Inicio; i ← tamaño(vet)	N2	¿i ≥ 2?
N3	j ← 1	N4	¿j ≤ i − 1?
N5	¿vet[j] > vet[j+1]?	N6	Intercambiar
N7	j ← j + 1	N8	i ← i − 1
N9	Fin		

Aristas: N1→N2; N2→N3/N9; N3→N4; N4→N5/N8; N5→N6/N7; N6→N7; N7→N4; N8→N2.

3. Actividad 2 | Complejidad ciclomática

Para un único componente conexo del grafo de flujo de control se emplea V(G) = E − N + 2P.

VARIABLE	VALOR	JUSTIFICACIÓN
E (aristas)	11	Conexiones dirigidas entre nodos
N (nodos)	9	N1, N2, ..., N9
P (componentes)	1	Un único subgrafo conectado
V(G)	11 − 9 + 2(1) = 4	Resultado de complejidad ciclomática

Verificación por predicados: existen tres nodos de decisión, N2, N4 y N5; por tanto V(G) = 3 + 1 = 4.

4. Actividad 3 | Caminos independientes

Se elige cobertura de decisiones + condición simple. Como cada decisión contiene un único predicado booleano, basta con obtener V y F en los nodos N2, N4 y N5. Los cuatro caminos se ejecutan con entradas factibles.

CAMINO	RASGO QUE LO DISTINGUE	ENTRADA
CP1	Salida directa N2→N9; no entra al for externo	[7]
CP2	Incluye N5→N7 (condición de intercambio F)	[1, 2]
CP3	Incluye N5→N6 (intercambio V)	[2, 1]
CP4	Repite el ciclo interno N7→N4 y externo N8→N2	[2, 3, 1]

Secuencias observadas de nodos

CP1 N1 → N2(F) → N9

CP2 N1 → N2(V) → N3 → N4(V) → N5(F) → N7 → N4(F) → N8 → N2(F) → N9

CP3 N1 → N2(V) → N3 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(F) → N9

CP4 N1 → N2(V) → N3 → N4(V) → N5(F) → N7 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(V) → N3 → N4(V) → N5(V) → N6 → N7 → N4(F) → N8 → N2(F) → N9

Los paréntesis (V/F) especifican la salida tomada en la visita a un nodo de decisión. La tabla CSV reproduce las cuatro rutas en el formato solicitado.

5. Actividad 4 | Diseño de casos de prueba

Las entradas y salidas de la tabla fueron verificadas con una implementación ejecutable del algoritmo. Los caminos corresponden al grafo de la figura 1.

ID CP	CAMINO EN EL GRAFO	VALORES DE PRUEBA	RESULTADO ESPERADO
CP1	N1→N2(F)→N9	[7]	[7]
CP2	N1→N2(V)→N3→N4(V)→N5(F)→N7→N4(F)→N8→N2(F)→N9	[1, 2]	[1, 2]
CP3	N1→N2(V)→N3→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(F)→N9	[2, 1]	[1, 2]
CP4	N1→N2(V)→N3→N4(V)→N5(F)→N7→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(V)→N3→N4(V)→N5(V)→N6→N7→N4(F)→N8→N2(F)→N9	[2, 3, 1]	[1, 2, 3]

6. Comprobación de cobertura y ejecución

DECISIÓN	SALIDAS OBSERVADAS	COBERTURA
N2: $i \geq 2$	V y F	2/2
N4: $j \leq i - 1$	V y F	2/2
N5: vet[j] > vet[j+1]	V y F	2/2
Total	6 de 6 salidas	100 %

Ejecución de ocho pruebas unitarias con unittest: CP1–CP4 y cuatro pruebas complementarias (lista vacía, duplicados, números negativos/cero y orden inverso). Resultado: 8 pruebas ejecutadas; 8 exitosas; 0 fallidas.

7. Conclusiones y reproducibilidad

La complejidad ciclomática de 4 justifica cuatro caminos de base. El conjunto CP1–CP4 cubre las salidas V y F de los tres predicados y alcanza el 100 % de cobertura de decisiones para el código modelado. La cobertura alcanzada no garantiza la ausencia total de defectos.

```
cd codigo
python -m unittest -v
```

Fuente principal: DSc. Edgar Sarmiento Calisaya, "Práctica 05: Generación de Pruebas Unitarias", Ingeniería de Software II, páginas identificadas como 5 y 6 en el documento suministrado. Entregables de apoyo: archivos DOT/PNG/SVG, CSV, implementación y registro de pruebas.